

SE446: Big Data Engineering

Big Data Challenge — News Pulse

Spring 2026 — Prof. Anis Koubaa

Format: 60-minute team challenge (teams of 2–3). Laptops only — no cluster, no GPU.

Allowed tools: ChatGPT, Claude, **VS Code + GitHub Copilot (Student Edition)**,

Stack Overflow, course notes, and any cloud LLM API.

Not allowed: “advanced vibe coding” — i.e. asking an agent (Cursor agent mode, Claude Code, Copilot Workspace, etc.) to autonomously write the whole pipeline for you. You must understand and type every line you submit; Copilot tab-completion is fine.

Grading: **no grading penalty** — this challenge is **bonus only**. A successful demo earns up to **+5 bonus points** on the course total. No submission, no penalty.

Submission: push your code to a **GitHub repository** (one repo per team) **by the end of the class hour**. Share the repository URL in the class channel before the demo round begins. Late pushes are not counted.

Deliverable: a 3-minute live demo of your dashboard + a one-paragraph insight.

The Story

You have been hired by a media-monitoring startup. Your job: turn raw news feeds into a single, glanceable “pulse” that shows what the world is talking about *right now*. This is a real big-data pattern (ingest → Spark transform → LLM summarisation → visualisation), shrunk to one laptop and one hour.

1 Pre-class Setup (mandatory)

You must have the following working *before* class. We will not debug installations during the challenge.

```
java -version          # must show 11 or 17
pip install pyspark==3.5.0 feedparser pandas streamlit requests
python -c "import pyspark, feedparser; print('ok')"
```

If you plan to use a cloud LLM, set your API key in your shell:

```
export ANTHROPIC_API_KEY=sk-...    # or OPENAI_API_KEY=sk-...
```

Watch out

PySpark requires the JDK. On macOS: `brew install openjdk@17`. On Ubuntu: `sudo apt install openjdk-17-jdk`. On Windows: install Adoptium Temurin 17 and add `JAVA_HOME` to your environment.

2 The Pipeline (what you must build)

Architecture (Structured Streaming, mandatory)

1. **Ingester (Python, separate process or thread):** pulls headlines from **at least 4 RSS feeds** every 30–60s and writes each batch as a JSON-lines file into `data/incoming/`.
2. **Spark Structured Streaming job:** reads `data/incoming/` as a file-source stream (`spark.readStream.json`) and produces three **streaming aggregations**: (a) headline count *per source*; (b) headline count per *1-hour tumbling window* on the timestamp; (c) the *top 10 keywords* across all headlines (after stop-word filtering). Output is written to **memory sinks** that Streamlit can query.
3. **LLM step:** call any LLM (cloud API or ChatGPT in the browser) on the current keyword snapshot and produce a **one-paragraph thematic summary**.
4. **Dashboard:** a **Streamlit** page with two charts (source mix + windowed volume) plus a panel showing the LLM summary, refreshing every 5–10 seconds.

Hard requirement: you *must* use `spark.readStream` and `writeStream`. A batch `spark.read` pipeline does not qualify.

3 Tasks

- T1. Ingester (10 pts).** A Python script that pulls 4+ RSS feeds with `feedparser` every 30–60s and **writes each batch as a JSON-lines file** into `data/incoming/`. Tolerate dead feeds (`try/except`). Each line is one record with fields (`source`, `title`, `url`, `ts`).
- T2. Spark Structured Streaming (25 pts).** Build a `SparkSession` in local mode and define a *stream* that watches `data/incoming/` via `spark.readStream.schema(...).json(...)`. Produce three streaming queries, each writing to a **memory sink**:
- `by_source`: `groupBy("source").count()` (output mode complete).
 - `by_window`: `tumbling window(ts, "1 hour")` aggregation with a watermark of e.g. `withWatermark("ts", "2 hours")`.
 - `top_words`: top 10 keywords (`regexp_extract_all` + `explode` + `lower`, stop-word + length filter, then `groupBy("word").count()`).
- Streamlit then queries those memory tables: `spark.sql("select * from by_source")`.
- T3. LLM summary (15 pts).** Send your top-15 keywords to an LLM (any provider) with a prompt of your design. Constrain it to one paragraph, no more than 80 words, and require it to mention at least three named storylines. Be defensive: if the API fails, fall back to a keyword-only summary so the dashboard never crashes.
- T4. Streamlit dashboard (25 pts).** A single-page Streamlit app with: a bar chart of `by_source`, a line chart of `by_hour`, a table of `top_words`, and a text box for the LLM summary. Run with `streamlit run app.py`.
- T5. Reflection (15 pts).** In one paragraph (max 100 words), answer: *which step of your pipeline would break first if the input grew 1000×, and which Spark feature would you reach for to fix it?*
- T6. Demo (10 pts).** Live 3-minute walkthrough. Show the dashboard refreshing on real RSS data and read your reflection. We grade on clarity, not polish.

Bonus (up to +10 pts)

- **GDELT slice.** Download one 15-minute GDELT events CSV from data.gdeltproject.org/gdeltv2/ and join it to your RSS data on a country/keyword field.
- **Refresh loop.** Make the dashboard auto-refresh every 60s.
- **Topic clustering.** Cluster the headlines into 3 topics with TF-IDF + KMeans from Spark MLlib.

4 Starter Guidelines & Sample Code

These are **minimum viable snippets**. Type them in, understand them, then extend. Do *not* paste these into an agent and ask it to “finish the project” — that’s the vibe-coding line.

These are **skeletons, not solutions**. The hard parts (TODOs) are yours. Anything missing in the skeletons is also yours.

(a) Ingester skeleton — one file per tick

```
# ingestor.py
import os, json, time
INCOMING = "data/incoming"
os.makedirs(INCOMING, exist_ok=True)

FEEDS = { ... } # TODO: 4+ (source -> url)

def pull_once(tick: int):
    rows = []
    # TODO: for each feed, use feedparser to parse it,
    # tolerate failures (try/except per feed),
    # and append dicts with keys (source, title, url, ts) to 'rows'.
    path = os.path.join(INCOMING, f"batch_{tick}.json")
    with open(path, "w") as f:
        for r in rows:
            f.write(json.dumps(r) + "\n") # JSONL: one record per line

if __name__ == "__main__":
    tick = 0
    while True:
        pull_once(tick); tick += 1; time.sleep(60)
```

(b) Structured Streaming skeleton

```
# streaming_job.py
from pyspark.sql import SparkSession, functions as F
from pyspark.sql.types import StructType, StructField, StringType,
    TimestampType

spark = (SparkSession.builder.appName("NewsPulse")
        .master("local[*]").getOrCreate())
spark.sparkContext.setLogLevel("ERROR")

schema = StructType([ ... ]) # TODO: source, title, url, ts (
    TimestampType)

stream = spark.readStream.schema(schema).json("data/incoming")
```

```
# Example query you can copy from --- writes a memory sink "by_source".
q_src = (stream.groupBy("source").count()
        .writeStream.outputMode("complete")
        .format("memory").queryName("by_source").start())

# TODO 1: streaming windowed query "by_window".
#         Apply withWatermark("ts", "...") and groupBy(F.window("ts", "1
#         hour"))
#         and write to a memory sink named "by_window".

# TODO 2: streaming top-words query "top_words".
#         Tokenise the title (regex_extract_all + explode),
#         filter stop-words and short tokens,
#         groupBy(word).count(), memory sink named "top_words".

spark.streams.awaitAnyTermination()
```

(c) Streamlit skeleton

```
# app.py
import streamlit as st
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("NewsPulse").getOrCreate()

st.title("News Pulse - live")
# TODO: query the three memory tables via spark.sql(...).toPandas()
# TODO: render a bar chart, a line chart, a top-words table, and the LLM
#       summary.
```

Hint

Run **three terminals**: (1) `python ingester.py`, (2) `python streaming_job.py` (keep alive), (3) `streamlit run app.py`. You can sanity-check the streaming job before writing the dashboard by calling `spark.sql("select * from by_source").show()` in a fourth terminal attached to the same session.

5 Tips

Hint

- Start by writing *one* JSON file by hand into `data/incoming/` and confirming `by_source` updates — you don't need the RSS ingester yet.
- Don't fight Spark's logs — call `sc.setLogLevel("ERROR")` once and forget.
- `outputMode("complete")` is required when using `groupBy` aggregations. Use `outputMode("append")` only if you switch to a windowed sink.
- Without a watermark, the streaming state grows unbounded. Add `.withWatermark("ts", "2 hours")` on any windowed query.
- For the LLM step, your prompt is the product. Iterate it 2–3 times before you optimise the chart layout.
- Streamlit reruns the whole script on every interaction — query the memory tables *inside* the script body so they refresh on every rerun.

- If file-source streaming feels exotic, remember it is just “Spark watches a folder; new file = new mini-batch.”

6 Bonus Scoring (no penalty)

This challenge is **not graded**. A working submission demoed live and pushed to GitHub before the class ends earns **bonus points on your course total**, distributed below. Skipping the challenge has **no penalty**.

Component	Bonus pts
Ingestion (T1) — live RSS, dead-feed tolerant	+0.5
Spark transform (T2) — all three aggregations	+1.5
LLM summary (T3) — grounded, length-bounded, with fallback	+0.5
Streamlit dashboard (T4) — runs locally, 2 charts + summary	+1.0
Reflection (T5) — correctly identifies scaling bottleneck	+0.5
Demo (T6) — live, on real data, in 3 minutes	+0.5
GitHub push completed before class ends	+0.5
Maximum bonus	+5.0

Watch out

Submission rule. Push your team’s code to a GitHub repository before class ends. Email or paste the repo URL in the class channel. *Pushes after the bell are not counted, even by one minute.* The point of the challenge is to ship under time pressure — this is what big-data engineering at work feels like.

7 What we are actually testing

- Can you stitch together a real ingest → transform → insight pipeline against *live* data?
- Do you reach for the right tool at each layer (Spark for aggregation, an LLM for narrative, Streamlit for delivery)?
- Can you reason about where your pipeline would scale poorly — which is the only question that distinguishes a data engineer from a scripter.

Good luck. Ship something that actually runs.